1. Queue as an Abstract Data Type

A **queue** is an ordered linear data structure that follows the **First In First Out (FIFO)** principle. Elements are inserted at one end (rear) and deleted from the other end (front).

Definition: A queue is a collection of elements where:

- Insertions (Enqueue) occur at the REAR end
- Deletions (Dequeue) occur at the FRONT end
- First element added is the first element removed

Real-world Analogies:

- Queue at ticket counter
- Print job scheduling
- Call center waiting line
- Checkout line at supermarket

2. Queue ADT Specification

```
structure QUEUE (item)
   declare CREATEQ() → queue
           ADDQ(item, queue) → queue
           DELETEQ(queue) → queue
           FRONT(queue) → item
           ISEMTQ(queue) → boolean;

   for all Q ∈ queue, i ∈ item let
      ISEMTQ(CREATEQ) ::= true
      ISEMTQ(ADDQ(i,Q)) ::= false
      DELETEQ(CREATEQ) ::= error
      DELETEQ(ADDQ(i,Q)) ::=
         if ISEMTQ(Q) then CREATEQ
         else ADDQ(i, DELETEQ(Q))
      FRONT(CREATEQ) ::= error
      FRONT(ADDQ(i,Q)) ::=
         if ISEMTQ(Q) then i
         else FRONT(Q)
end
end QUEUE
```

Operations Defined:

- **CREATEQ()**: Creates an empty queue

- **ADDQ(i,Q)**: Adds element i to rear of queue Q
- **DELETEQ(Q)**: Removes front element from queue Q
- **FRONT(Q)**: Returns front element without removing it
- **ISEMTQ(Q)**: Returns true if queue is empty

3. Basic Queue Operations

Essential Operations:

1. **Enqueue (AddQ)**: Insert element at rear
2. **Dequeue (DeleteQ)**: Remove element from front
3. **PeepFront**: View front element
4. **PeepRear**: View rear element
5. **isEmpty**: Check if queue is empty
6. **isFull**: Check if queue is full

4. Sequential Organization Using Arrays

Implementation Strategy: Queue is represented using a one-dimensional array with two pointers:

- **front**: Points to the first element (for deletion)
- **rear**: Points to the last element (for insertion)

Structure Definition:

```c
#define MAX_SIZE 100

typedef struct {
    int items[MAX_SIZE];
    int front;
    int rear;
} Queue;
```

Initial State:

```
front = -1
rear = -1
```

Memory Representation:

```
Index: 0   1   2   3   4   5
     [ ] [ ] [ ] [ ] [ ] [ ]
front = -1
rear = -1
```

5. Array-Based Queue Operations

5.1 isEmpty() Operation

```c
int isEmpty(Queue *q) {
    if (q->rear == q->front)
        return 1; // true
    else
        return 0; // false
}
```

Alternative: `front == -1` or `front == rear`

5.2 isFull() Operation

```c
int isFull(Queue *q) {
    if (q->rear == MAX_SIZE - 1)
        return 1; // true
    else
        return 0; // false
}
```

5.3 Enqueue (AddQ) Operation

Algorithm:

```
Algorithm AddQ(q[], elem)
{
    if (isFull())
        print "Queue is Full"
    else
    {
        rear = rear + 1;
        q[rear] = elem;
    }
}
```

C Implementation:

```c
void enqueue(Queue *q, int elem) {
    if (isFull(q)) {
        printf("Queue Overflow\n");
        return;
    }

    if (q->front == -1)  // First element
        q->front = 0;

    q->rear++;
    q->items[q->rear] = elem;
    printf("Enqueued: %d\n", elem);
}
```

Time Complexity: O(1)

5.4 Dequeue (DeleteQ) Operation

Algorithm:

```
Algorithm DelQ(q[])
{
    if (isEmpty())
        return -1
    else
    {
        front = front + 1;
        elem = q[front];
        return elem;
    }
}
```

C Implementation:

```c
```

```c
int dequeue(Queue *q) {
    if (isEmpty(q)) {
        printf("Queue Underflow\n");
        return -1;
    }

    q->front++;
    int elem = q->items[q->front];

    // Reset queue when empty
    if (q->front == q->rear) {
        q->front = q->rear = -1;
    }

    printf("Dequeued: %d\n", elem);
    return elem;
}
```

Time Complexity: O(1)

6. Step-by-Step Queue Operations Example

Sequence: AddQ(11), AddQ(22), AddQ(33), AddQ(44), DelQ(), DelQ(), DelQ()

```
Initial State:
Array: [  ] [  ] [  ] [  ]
       0   1   2   3
Front = -1, Rear = -1
Queue is EMPTY
---

Operation: AddQ(11)
Action: rear = 0, q[0] = 11
Array: [11] [  ] [  ] [  ]
       0   1   2   3
       ↑
      F,R
Front = 0, Rear = 0
---

Operation: AddQ(22)
Action: rear = 1, q[1] = 22
Array: [11] [22] [  ] [  ]
       0   1   2   3
       ↑   ↑
       F   R
Front = 0, Rear = 1
---

Operation: AddQ(33)
Action: rear = 2, q[2] = 33
Array: [11] [22] [33] [  ]
       0   1   2   3
       ↑       ↑
       F       R
Front = 0, Rear = 2
---

Operation: AddQ(44)
Action: rear = 3, q[3] = 44
Array: [11] [22] [33] [44]
       0   1   2   3
       ↑           ↑
       F           R
Front = 0, Rear = 3
Queue is FULL
---

Operation: DelQ()
Action: front = 1, elem = q[0] = 11
```

```
Array: [  ] [22] [33] [44]
      0   1   2   3
          ↑       ↑
          F       R
Front = 1, Rear = 3
Deleted: 11
---

Operation: DelQ()
Action: front = 2, elem = q[1] = 22
Array: [  ] [  ] [33] [44]
      0   1   2   3
              ↑   ↑
              F   R
Front = 2, Rear = 3
Deleted: 22
---

Operation: DelQ()
Action: front = 3, elem = q[2] = 33
Array: [  ] [  ] [  ] [44]
      0   1   2   3
                  ↑
                 F,R
Front = 3, Rear = 3
Deleted: 33
```

7. Problem with Simple Array Queue

Major Issue: **Memory Wastage** - Once elements are dequeued, space at the beginning becomes unusable.

Example Problem:

```
After several enqueue/dequeue operations:
Array: [  ] [  ] [  ] [78] [56]
      0   1   2   3   4
                  ↑   ↑
                  F   R

Trying to AddQ(34):
- rear = 4 (last index)
- isFull() returns true
- But positions 0, 1, 2 are FREE!
- Queue appears full but has empty space
```

Solution: Circular Queue

8. Circular Queue Concept

Definition: A circular queue treats the array as circular - when rear reaches the end, it wraps around to the beginning if space is available at the front.

Visualization:

```
Linear View:          Circular View:
[0] [1] [2] [3]

                        [0]
                       /   \
                    [3]     [1]
                       \   /
                        [2]
```

Key Idea: `(rear + 1) % MAX_SIZE` gives next position circularly

Expression for Circular Movement:

```
i = (i + 1) % MAX_SIZE

Examples:
0: (0 + 1) % 5 = 1
1: (1 + 1) % 5 = 2
2: (2 + 1) % 5 = 3
3: (3 + 1) % 5 = 4
4: (4 + 1) % 5 = 0  ← Wraps around!
```

9. Circular Queue Implementation

9.1 Initial State

```c
front = 0
rear = 0
```

Note: Unlike simple queue, circular queue starts with front = rear = 0

9.2 Full Condition

Circular Queue is Full when:

```
(rear + 1) % MAX_SIZE == front
```

Why this condition?

- One position is always kept empty to distinguish full from empty
- If we fill all positions, full and empty would look the same

9.3 Empty Condition

```c
front == rear  // Queue is empty
```

9.4 AddCQ (Circular Queue Enqueue)

Algorithm:

```
Algorithm AddCQ(elem)
{
   // Q[0..n-1] is the circular queue
   // rear points to last item
   // front is one position counter-clockwise from first

   if ((rear + 1) % n == front)
      print "Queue Full"
   else
   {
      rear = (rear + 1) % n;
      Q[rear] = elem;
   }
}
```

C Implementation:

```c
```

```c
void enqueueCircular(Queue *q, int elem) {
    if ((q->rear + 1) % MAX_SIZE == q->front) {
        printf("Circular Queue Full\n");
        return;
    }

    q->rear = (q->rear + 1) % MAX_SIZE;
    q->items[q->rear] = elem;
    printf("Enqueued: %d\n", elem);
}
```

9.5 DelCQ (Circular Queue Dequeue)

Algorithm:

```
Algorithm DelCQ(elem)
{
    // Removes the front element

    if (front == rear)
        print "Queue Empty"
    else
    {
        front = (front + 1) % n;
        elem = Q[front];
        return elem;
    }
}
```

C Implementation:

c

```c
int dequeueCircular(Queue *q) {
    if (q->front == q->rear) {
        printf("Circular Queue Empty\n");
        return -1;
    }

    q->front = (q->front + 1) % MAX_SIZE;
    int elem = q->items[q->front];
    printf("Dequeued: %d\n", elem);
    return elem;
}
```

10. Circular Queue Example

Array Size: 5 (indices 0-4) **Sequence**: Add(10), Add(20), Add(30), Del(), Del(), Add(40), Add(50), Add(60)

Initial: front = 0, rear = 0
Array: [ ] [ ] [ ] [ ] [ ]
0 1 2 3 4
↑
F,R

Add(10):
rear = (0+1)%5 = 1
Array: [ ] [10] [ ] [ ] [ ]
0 1 2 3 4
↑ ↑
F R

Add(20):
rear = (1+1)%5 = 2
Array: [ ] [10] [20] [ ] [ ]
0 1 2 3 4
↑ ↑
F R

Add(30):
rear = (2+1)%5 = 3
Array: [ ] [10] [20] [30] [ ]
0 1 2 3 4
↑ ↑
F R

Del():
front = (0+1)%5 = 1
Removed: 10
Array: [ ] [ ] [20] [30] [ ]
0 1 2 3 4
↑ ↑
F R

Del():
front = (1+1)%5 = 2
Removed: 20
Array: [ ] [ ] [ ] [30] [ ]
0 1 2 3 4
↑ ↑

```
          F  R
---

Add(40):
rear = (3+1)%5 = 4
Array: [  ] [  ] [  ] [30] [40]
      0   1   2   3   4
          ↑       ↑
          F       R
---

Add(50):
rear = (4+1)%5 = 0  ← Wraps around!
Array: [50] [  ] [  ] [30] [40]
      0   1   2   3   4
      ↑       ↑
      R       F
---

Add(60):
rear = (0+1)%5 = 1
Array: [50] [60] [  ] [30] [40]
      0   1   2   3   4
        ↑   ↑
        R   F
---

Trying Add(70):
Check: (1+1)%5 = 2 == 2 (front)
Result: Queue Full!
(One position kept empty)
```

11. Advantages of Circular Queue

Over Simple Array Queue:

1. **Efficient Memory Utilization**:
   - Reuses freed space at front
   - No memory wastage
   - All positions can be used
2. **No Need to Shift**:
   - Elements don't need to be shifted
   - Constant time operations maintained
3. **Better Space Complexity**:

- Can store n-1 elements in array of size n
- Simple queue wastes front positions

4. **Suitable for Fixed-Size Buffers**:
   - Ideal for circular buffers
   - Ring buffers in operating systems
   - I/O buffers

Comparison:

Aspect	**Simple Queue**	**Circular Queue**
Memory Usage	Wasteful	Efficient
Space Reuse	No	Yes
Capacity	Degrades over time	Constant
Implementation	Simpler	Slightly complex
Real-world use	Limited	Widespread

12. Complete Circular Queue Program

```c
```

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 5

typedef struct {
    int items[MAX_SIZE];
    int front;
    int rear;
} CircularQueue;

void initQueue(CircularQueue *q) {
    q->front = 0;
    q->rear = 0;
}

int isEmpty(CircularQueue *q) {
    return (q->front == q->rear);
}

int isFull(CircularQueue *q) {
    return ((q->rear + 1) % MAX_SIZE == q->front);
}

void enqueue(CircularQueue *q, int elem) {
    if (isFull(q)) {
        printf("Queue Full\n");
        return;
    }
    q->rear = (q->rear + 1) % MAX_SIZE;
    q->items[q->rear] = elem;
    printf("Enqueued: %d\n", elem);
}

int dequeue(CircularQueue *q) {
    if (isEmpty(q)) {
        printf("Queue Empty\n");
        return -1;
    }
    q->front = (q->front + 1) % MAX_SIZE;
    int elem = q->items[q->front];
    printf("Dequeued: %d\n", elem);
    return elem;
}

void display(CircularQueue *q) {
```

```
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue elements: ");
    int i = (q->front + 1) % MAX_SIZE;
    while (i != (q->rear + 1) % MAX_SIZE) {
        printf("%d ", q->items[i]);
        i = (i + 1) % MAX_SIZE;
    }
    printf("\n");
}

int main() {
    CircularQueue q;
    initQueue(&q);

    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);
    display(&q);

    dequeue(&q);
    display(&q);

    enqueue(&q, 40);
    enqueue(&q, 50);
    display(&q);

    return 0;
}
```

13. Time and Space Complexity

All Operations:

Operation	Time Complexity	Space Complexity
Enqueue	O(1)	O(1)
Dequeue	O(1)	O(1)
isEmpty	O(1)	O(1)
isFull	O(1)	O(1)
Display	O(n)	O(1)

Overall Space: O(n) where n = MAX_SIZE

14. Applications of Queues

Common Applications:

1. **CPU Scheduling**:
   - Round-robin scheduling
   - Process management
   - Job queues
2. **I/O Buffers**:
   - Keyboard buffer
   - Printer spooling
   - Disk scheduling
3. **Network Applications**:
   - Router packet queues
   - Network traffic management
   - Request handling
4. **Real-World Systems**:
   - Call center systems
   - Ticket booking systems
   - Order processing
5. **BFS Algorithm**:
   - Graph traversal
   - Shortest path finding
   - Level-order tree traversal

15. Job Scheduling Example

Operating System Process Scheduling:

```
Job Queue: Jobs awaiting CPU time
Ready Queue: Processes in main memory, ready to execute
Device Queues: Processes waiting for I/O devices

Example Flow:
1. New jobs enter Job Queue
2. Selected jobs move to Ready Queue
3. CPU scheduler picks from Ready Queue
4. If I/O needed, process moves to Device Queue
5. After I/O, returns to Ready Queue
```

Queue Types in OS:

1. **Job Queue**: All processes in system
2. **Ready Queue**: Processes ready to execute
3. **Device Queue**: Processes waiting for I/O

Q7: Explain linked queue implementation, deque (double-ended queue) with its types, and complete implementations with examples.

Answer:

1. Linked Queue Implementation

A **linked queue** uses a singly linked list to implement queue operations. It uses two pointers: **front** and **rear**.

Advantages over Array Queue:

- Dynamic size - no overflow (until memory exhausted)
- No memory wastage
- No need for circular logic
- Efficient insertion and deletion

1.1 Structure Definition

```c
```

```
struct node {
    int data;
    struct node *next;
};

struct node *front, *rear;
```

Visual Representation:

Empty Queue:

```
front->next = NULL
rear = NULL
```

Queue with Elements:

```
front → [H|•] → [10|•] → [20|•] → [30|NULL]
                  ↑
                rear
```

1.2 Initialization

```c
void initQueue() {
    front = (struct node *) malloc(sizeof(struct node));
    front->next = NULL;
    rear = NULL;
}
```

1.3 isEmpty Operation

```c
int isEmpty() {
    if (front->next == NULL)
        return 1;  // true
    else
        return 0;  // false
}
```

1.4 Enqueue Operation

Algorithm:

```
Algorithm enqueue(element)
{
   // Allocate memory for new node
   new_node = malloc(sizeof(struct node));
   new_node->data = element;
   new_node->next = NULL;

   if (isEmpty())
   {
      // First element
      front->next = new_node;
      rear = new_node;
   }
   else
   {
      // Add at rear
      rear->next = new_node;
      rear = new_node;
   }
}
```

C Implementation:

```c
```

```c
void enqueue(int element) {
    struct node *new_node;

    // Allocate memory
    new_node = (struct node *) malloc(sizeof(struct node));
    if (new_node == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

    // Initialize new node
    new_node->data = element;
    new_node->next = NULL;

    if (isEmpty()) {
        front->next = new_node;
        rear = new_node;
    } else {
        rear->next = new_node;
        rear = new_node;
    }

    printf("Enqueued: %d\n", element);
}
```

Visual Steps:

```
Step 1: Empty Queue
front → [H|NULL]
rear = NULL

Step 2: Enqueue(10)
front → [H|•] → [10|NULL]
                  ↑
                rear

Step 3: Enqueue(20)
front → [H|•] → [10|•] → [20|NULL]
                         ↑
                       rear

Step 4: Enqueue(30)
front → [H|•] → [10|•] → [20|•] → [30|NULL]
                                  ↑
                                rear
```

Time Complexity: O(1)

1.5 Dequeue Operation

Algorithm:

```
Algorithm dequeue()
{
   if (isEmpty())
   {
      print "Queue Empty";
      return -1;
   }
   else
   {
      temp = front->next;
      value = temp->data;
      front->next = temp->next;
      temp->next = NULL;
      free(temp);
      return value;
   }
}
```

C Implementation:

```c
```

```c
int dequeue() {
    struct node *temp;
    int value;

    if (isEmpty()) {
        printf("Queue Underflow\n");
        return -1;
    }

    temp = front->next;
    value = temp->data;
    front->next = temp->next;

    // If queue becomes empty, update rear
    if (front->next == NULL) {
        rear = NULL;
    }

    free(temp);
    printf("Dequeued: %d\n", value);
    return value;
}
```

Visual Steps:

```
Before Dequeue:
front → [H|•] → [10|•] → [20|•] → [30|NULL]
          ↑              ↑
        temp           rear

After Dequeue (removed 10):
front → [H|•] → [20|•] → [30|NULL]
                  ↑
                rear
```

Time Complexity: O(1)

2. Double-Ended Queue (Deque)

Definition: A deque (pronounced "deck") is a generalized queue where elements can be added or removed from **either the front or rear**.

Characteristics:

- Hybrid structure combining stack and queue capabilities

- Provides all stack and queue operations
- More flexible than standard queue

Operations:

- **pushFront()**: Insert element at front
- **pushRear()**: Insert element at rear
- **popFront()**: Remove element from front
- **popRear()**: Remove element from rear
- **isEmpty()**: Check if deque is empty
- **isFull()**: Check if deque is full (array implementation)

3. Types of Deque

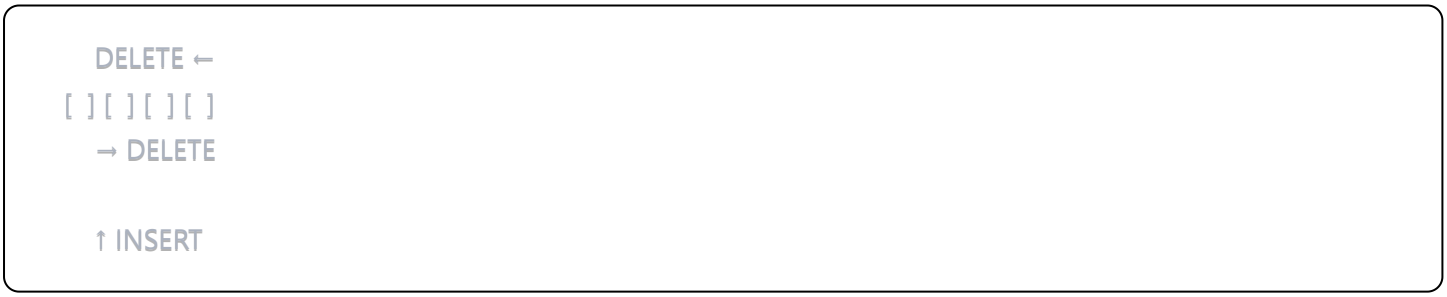
3.1 Input-Restricted Deque

Definition: Insertion can be made at **one end only** (rear), but deletion can be made from **both ends** (front and rear).

Operations Allowed:

- Insert: Rear only
- Delete: Front or Rear

Visual:



Use Cases:

- Undo-redo with limited input
- Priority scheduling with front removal

3.2 Output-Restricted Deque

Definition: Insertion can be made at **both ends** (front and rear), but deletion can be made from **one end only** (front).

Operations Allowed:

- Insert: Front or Rear

- Delete: Front only

Visual:

```
    DELETE ←
 [ ][ ][ ][ ]

  ↑         ↑
INSERT    INSERT
```

Use Cases:

- Job scheduling with priority insertion
- Task management systems

4. Deque Implementation Using Array

```c

```

```c
#define MAX 100

typedef struct {
    int items[MAX];
    int front;
    int rear;
} Deque;

void initDeque(Deque *dq) {
    dq->front = -1;
    dq->rear = -1;
}

int isEmpty(Deque *dq) {
    return (dq->front == -1);
}

int isFull(Deque *dq) {
    return ((dq->front == 0 && dq->rear == MAX - 1) ||
            (dq->front == dq->rear + 1));
}

// Insert at front
void pushFront(Deque *dq, int elem) {
    if (isFull(dq)) {
        printf("Deque Full\n");
        return;
    }

    if (dq->front == -1) {  // First element
        dq->front = dq->rear = 0;
    } else if (dq->front == 0) {
        dq->front = MAX - 1;  // Wrap around
    } else {
        dq->front--;
    }

    dq->items[dq->front] = elem;
    printf("Pushed %d at front\n", elem);
}

// Insert at rear
void pushRear(Deque *dq, int elem) {
    if (isFull(dq)) {
        printf("Deque Full\n");
        return;
```

```
    }

    if (dq->front == -1) {  // First element
        dq->front = dq->rear = 0;
    } else if (dq->rear == MAX - 1) {
        dq->rear = 0;  // Wrap around
    } else {
        dq->rear++;
    }

    dq->items[dq->rear] = elem;
    printf("Pushed %d at rear\n", elem);
}

// Delete from front
int popFront(Deque *dq) {
    if (isEmpty(dq)) {
        printf("Deque Empty\n");
        return -1;
    }

    int elem = dq->items[dq->front];

    if (dq->front == dq->rear) {  // Only one element
        dq->front = dq->rear = -1;
    } else if (dq->front == MAX - 1) {
        dq->front = 0;  // Wrap around
    } else {
        dq->front++;
    }

    printf("Popped %d from front\n", elem);
    return elem;
}

// Delete from rear
int popRear(Deque *dq) {
    if (isEmpty(dq)) {
        printf("Deque Empty\n");
        return -1;
    }

    int elem = dq->items[dq->rear];

    if (dq->front == dq->rear) {  // Only one element
        dq->front = dq->rear = -1;
    } else if (dq->rear == 0) {
```

```
        dq->rear = MAX - 1;  // Wrap around
    } else {
        dq->rear--;
    }

    printf("Popped %d from rear\n", elem);
    return elem;
}
```

5. Deque Example Execution

Sequence: pushFront('a'), pushFront('b'), pushRear('c'), popFront(), popRear()

```
Operation | Deque Content       | Front | Rear
----------|---------------------|-------|------
Initial   | []                  | -1    | -1
pushFront('a') | ['a']          | 0     | 0
pushFront('b') | ['b', 'a']     | 4*    | 0
pushRear('c')  | ['b', 'a', 'c']| 4     | 1
popFront()     | ['a', 'c']     | 0     | 1
popRear()      | ['a']          | 0     | 0

* Assuming circular array with wrap-around
```

6. Applications of Deque and Queues

Deque Applications:

1. **Browser History**: Forward and backward navigation
2. **Undo-Redo Operations**: Both ends accessible
3. **Sliding Window Problems**: Add/remove from both ends
4. **A-Steal Job Scheduling**: Work stealing algorithms
5. **Palindrome Checking**: Compare from both ends

General Queue Applications:

1. **Job Scheduling**: OS process management
2. **Print Spooling**: Printer queue management
3. **BFS Algorithm**: Level-order traversal
4. **Network Packet Management**: Router queues
5. **Call Center Systems**: Customer service queues

7. Job Scheduling in Operating Systems

Process Scheduling Queues:

1. **Job Queue**: All processes in the system

```
New Process → Job Queue → Long-term Scheduler
```

2. **Ready Queue**: Processes in main memory, ready to execute

```
Job Queue → Ready Queue → CPU
```

3. **Device Queues**: Processes waiting for I/O

```
CPU → I/O Request → Device Queue → I/O Complete → Ready Queue
```

Example Flow:

```
New jobs arrive → Job Queue
                    ↓
          Long-term Scheduler selects
                    ↓
               Ready Queue
                    ↓
          Short-term Scheduler selects
                    ↓
                  CPU
                    ↓
          ┌─────────┴─────────┐
          ↓                   ↓
    I/O Request         Time Slice Expired
          ↓                   ↓
    Device Queue          Ready Queue
```

8. Comparison Summary

Feature	Simple Queue	Circular Queue	Linked Queue	Deque
Size	Fixed	Fixed	Dynamic	Fixed/Dynamic
Memory	Wasteful	Efficient	Efficient	Moderate
Insertion	Rear	Rear	Rear	Both ends
Deletion	Front	Front	Front	Both ends
Complexity	Simple	Moderate	Moderate	Complex